KOENIG

step forward



Configuration Management

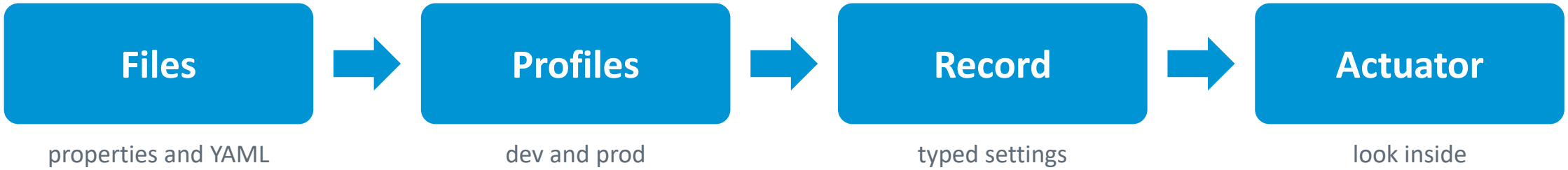
Spring Boot Fundamentals

What You Will Learn

1. application.properties and YAML
2. Profiles and profile files
3. @ConfigurationProperties records
4. Validating configuration
5. Actuator: env, configprops, health, info
6. Property placeholders and Handling secrets

Demo Scenario: Learnly Platform Settings

Module 8 built the Learnly student API. Now its settings must change per environment without a rebuild, wrong values must stop it early, and passwords must stay secret.



Keeping Learnly safe from

- **Rebuilds**
- **Wrong values**
- **Leaked secrets**

Why Externalise Configuration?

- **Same jar,** different settings per environment.
- **No rebuild** to change a mail server or a port.
- **Safer:** passwords stay out of source code.
- **Rule:** config lives in the environment.

Code
built once, never edited per server

Configuration
dev | test | prod values

Secrets
passwords, tokens, keys

Rule of thumb: if it changes between environments, it is not code

application.properties Basics

- **Location:** src/main/resources.
- **Format:** one key=value per line.
- **Keys:** dotted names, lower case, dashes.
- **Comments** start with #.
- **Boot's own** settings and yours live together.

```
# Boot's own settings
spring.application.name=learnly
server.port=8080

# Our own settings (any prefix we like)
learnly.mail.host=localhost
learnly.mail.port=25
learnly.mail.retry-count=3
learnly.mail.enabled=true

learnly.datasource.url=jdbc:mysql://localhost:3306/learnlydb
learnly.datasource.pool-size=5
```

The Same Settings in YAML

- **YAML:** indentation shows the hierarchy.
- **File:** application.yaml or .yml.
- **Less repetition** for deeply nested keys.
- **Careful:** spaces only, never tabs.
- **This course** uses .properties for the demos.

```yaml
spring:
  application:
    name: learnly
server:
  port: 8080
app:
  mail:
    host: localhost
    port: 25
    retry-count: 3
    enabled: true
  datasource:
    url: jdbc:mysql://localhost:3306/learnlydb
    pool-size: 5
```

Properties or YAML?

.properties

Flat, easy to grep and diff. No indentation traps. Every line is complete on its own. Verbose when keys are nested deeply.

.yaml

Compact and readable for hierarchies and lists. Indentation errors are easy to make. Supports multiple documents in one file.

- **If both exist** in the same folder, .properties wins over YAML.
- **Best practice:** choose one format and stay with it.

Demo: Properties Files and Where Boot Looks

What we build

- **application.properties:** Boot's settings and ours.
- **YAML:** same keys, the properties file wins.
- **./config:** overrides the packaged file.

How the concept helps

- **Defaults:** ship inside the jar.
- **Override:** operators change values, no rebuild.
- **One format:** avoid mixing YAML and properties.

What Is a Spring Profile?

- **Profile:** a named set of settings (and beans).
- **Typical names:** dev, test, prod.
- **Active profile** chosen at startup, not in code.
- **Several** profiles can be active together.
- **None active?** the base file only.

dev - my laptop, local DB

test - shared test server

prod - real customers

same code, same jar

Analogy: a TV picture mode - Movie, Sport, Night - one TV, three settings

Profile-Specific Files

- **Naming rule:** application-{profile}.properties.
- **Base file:** only shared defaults.
- **Profile file:** only what differs.
- **Profile file wins** over the base file.
- **Same for YAML:** application-dev.yaml.

```
# application.properties (base)
learnly.mail.port=25
learnly.mail.retry-count=3

# application-dev.properties
learnly.mail.port=1025

# application-prod.properties
learnly.mail.port=587
learnly.mail.retry-count=5

# Result with profile prod:
#   port = 587, retry-count = 5
# Result with no profile:
#   port = 25,  retry-count = 3
```

Activating a Profile

- **Property:** spring.profiles.active.
- **Command line:** --spring.profiles.active=dev.
- **Environment:** SPRING_PROFILES_ACTIVE=prod.
- **IntelliJ:** Active profiles field.
- **Several:** comma separated, last wins.

```
# Gradle
./gradlew bootRun --args="--spring.profiles.active=dev"

# Jar
java -jar learnly.jar --spring.profiles.active=prod

# Environment variable (bash)
SPRING_PROFILES_ACTIVE=prod java -jar learnly.jar

# Two profiles: dev settings, then inspect on top
--spring.profiles.active=dev,inspect
```

Do not hard-code spring.profiles.active=prod in the base file

Profile Groups and @Profile

- **@Profile** puts a bean in only for a profile.
- **Group:** one name switches on several.
- **Negation:** @Profile("!prod").
- **Use sparingly:** config values beat if-profile code.

```
@Configuration
@Profile("safe")
public class SecretsSanitizerConfig {
    // beans only when 'safe' is active
}

# application.properties
spring.profiles.group.local=dev,inspect

# one switch, two profiles
--spring.profiles.active=local
```

Demo: Profiles

What we build

- **dev and prod files:** only what differs.
- **Active profiles:** dev, prod, both, a group.
- **@Profile:** a bean only when safe is on.

How the concept helps

- **Same jar:** different settings per environment.
- **Last wins:** the order of the active profiles.
- **Group:** one switch for several profiles.

Reading a Value with @Value

- **@Value:** injects one property by name.
- **Simple** for one or two values.
- **Weak:** typo = error only at runtime.
- **Scattered:** same key string in many places.
- **No grouping,** no validation.

```java
@Component
public class ConfigPrinter {

    private final String sender;

    public ConfigPrinter(
            @Value("${learnly.mail.sender}") String sender) {
        this.sender = sender;
    }

    // with a default value:
    // @Value("${learnly.mail.port:25}") int port
}
```

@ConfigurationProperties Records

- **One record** per group of properties.
- **prefix** selects the group.
- **Typed:** int, boolean, Duration, lists.
- **Immutable:** values set in the constructor.
- **Relaxed:** retry-count to retryCount.

```java
@ConfigurationProperties(prefix = "learnly.mail")
public record MailProperties(
        String host,
        int port,
        String sender,
        int retryCount,
        boolean enabled) {
}

// learnly.mail.host=localhost
// learnly.mail.port=25
// learnly.mail.retry-count=3
```

Boot converts text to the right type and maps the names for you

Relaxed Binding Names

Where	Form of learnly.mail.retry-count
.properties / .yaml	learnly.mail.retry-count (recommended)
camelCase	learnly.mail.retryCount
Underscore	learnly.mail.retry_count
Environment variable	LEARNLY_MAIL_RETRYCOUNT

For retry-count, both LEARNLY_MAIL_RETRYCOUNT and LEARNLY_MAIL_RETRY_COUNT bind

Registering and Injecting Records

- **@ConfigurationPropertiesScan** on the main class finds all records.
- **Or:** @EnableConfigurationProperties(X.class).
- **Inject** like any bean, by constructor.
- **Never** new MailProperties() yourself.

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class ConfigLabApp { ... }

@Service
public class NotificationService {

    private final MailProperties mail;

    public NotificationService(MailProperties mail) {
        this.mail = mail;
    }

    public String send(String to, String text) {
        return "via " + mail.host() + ":" + mail.port();
    }
}
```

Demo: @ConfigurationProperties Records

What we build

- **@Value:** one value, the old way.
- **MailProperties:** host, port, sender, retries.
- **Relaxed names:** LEARNLY_MAIL_RETRYCOUNT binds.

How the concept helps

- **Typed:** int and boolean, not strings.
- **Grouped:** one object per prefix.
- **Testable:** inject it by constructor.

Validating Configuration

- **@Validated** on the record turns checks on.
- **Constraints:** jakarta.validation annotations.
- **Starter:** spring-boot-starter-validation.
- **Checked** once, while the app starts.
- **Fail fast:** bad config never reaches users.

```
@ConfigurationProperties(prefix = "learnly.mail")
@Validated
public record MailProperties(
        @NotBlank String host,
        @Min(1) @Max(65535) int port,
        @NotBlank @Email String sender,
        @Min(0) @Max(10) int retryCount,
        boolean enabled) {
}

// imports: jakarta.validation.constraints.*
// @Validated: org.springframework.validation.annotation
```

When Validation Fails

```
APPLICATION FAILED TO START

Description:
Binding to target ...MailProperties failed:

    Property: learnly.mail.port
    Value: "99999"
    Origin: "learnly.mail.port" from property source "commandLineArgs"
    Reason: must be less than or equal to 65535

Action:
Update your application's configuration
```

- **Property** names the key.
- **Value** what was supplied.
- **Origin** tells you where it came from!
- **Reason** the broken rule.

A clear startup failure beats a bug found by customers

Demo: Validating Configuration

What we build

- **Constraints:** @NotBlank, @Min, @Max, @Email.
- **Port 99999:** the application will not start.
- **Bad sender:** same, another rule.

How the concept helps

- **Fail fast:** at startup, not in production.
- **Clear report:** property, value, origin, reason.
- **One place:** the rules sit on the record.

Spring Boot Actuator

- **Actuator:** a control window into a running app.
- **Dependency:** spring-boot-starter-actuator.
- **Endpoints** live under /actuator.
- **Read-only mostly:** health, info, env, configprops.
- **Must be protected** in production.

/actuator/health	Is the app UP?
/actuator/info	Name, version, build
/actuator/env	All property sources
/actuator/configprops	Bound @ConfigurationProperties

Exposing Endpoints Safely

- **Default:** only health is exposed over HTTP.
- **include** lists endpoint ids to open; exclude always wins.
- **Dev:** open more. Prod: open least.
- **Use a profile** (inspect) to switch the windows on.

```
# application.properties (safe default)
management.endpoints.web.exposure.include=health

# application-inspect.properties
management.endpoints.web.exposure.include=health,info,env,configprops
management.endpoint.env.show-values=ALWAYS
management.endpoint.configprops.show-values=ALWAYS
management.endpoint.health.show-details=always
```

Not exposed means 404, not 'forbidden'

Inspecting /env and /configprops

- **/actuator/env** lists every property source in priority order.
- **/env/{key}** shows the winner and its source.
- **/configprops** shows our bound records.
- **activeProfiles** at the top of /env.

```
GET /actuator/env/learnly.mail.port

{
  "property": {
    "source": "Config resource ...application-dev.properties",
    "value": "1025"
  },
  "propertySources": [
    { "name": "commandLineArgs" },
    { "name": "systemEnvironment" },
    { "name": "Config resource ...application-dev.properties",
      "property": { "value": "1025" } },
    { "name": "Config resource ...application.properties",
      "property": { "value": "25" } }
  ]
}
```

Masking Sensitive Values

- **show-values** NEVER: all values are ******.
- **ALWAYS:** everyone sees values.
- **WHEN_AUTHORIZED:** only logged-in roles.
- **Default:** NEVER.
- **SanitizingFunction:** hide only chosen keys.

```
@Bean
SanitizingFunction secretsSanitizer() {
    return SanitizingFunction.sanitizeValue()
        .ifKeyContains("password", "secret");
}

// GET /actuator/env/learnly.mail.password
// -> "value": "******"
```

Masking is the last line of defence: keep secrets out of files too

Health and Info Endpoints

- **/health** UP or DOWN, used by load balancers.
- **show-details** never | when-authorized | always.
- **Probes:** /health/liveness, /health/readiness.
- **/info** shows info.* properties.
- **Enable env info:** management.info.env.enabled=true.

```
GET /actuator/health
{ "status": "UP" }

GET /actuator/info
{
  "app": {
    "name": "learnly",
    "description": "Learnly platform settings"
  }
}

# application-inspect.properties
management.info.env.enabled=true
info.app.name=${spring.application.name}
```

Demo: Actuator

What we build

- **health:** open by default, probes included.
- **inspect profile:** opens env, configprops, info.
- **Masking:** a SanitizingFunction in profile safe.

How the concept helps

- **Winner:** env shows the value and its source.
- **Safe default:** everything else is a 404.
- **Masked:** passwords show as ******.

Placeholders and Default Values

- **Syntax:** ${NAME:default}.
- **Reads** another property or an env variable.
- **After the colon:** used when the name is missing.
- **No default** and missing = startup error.

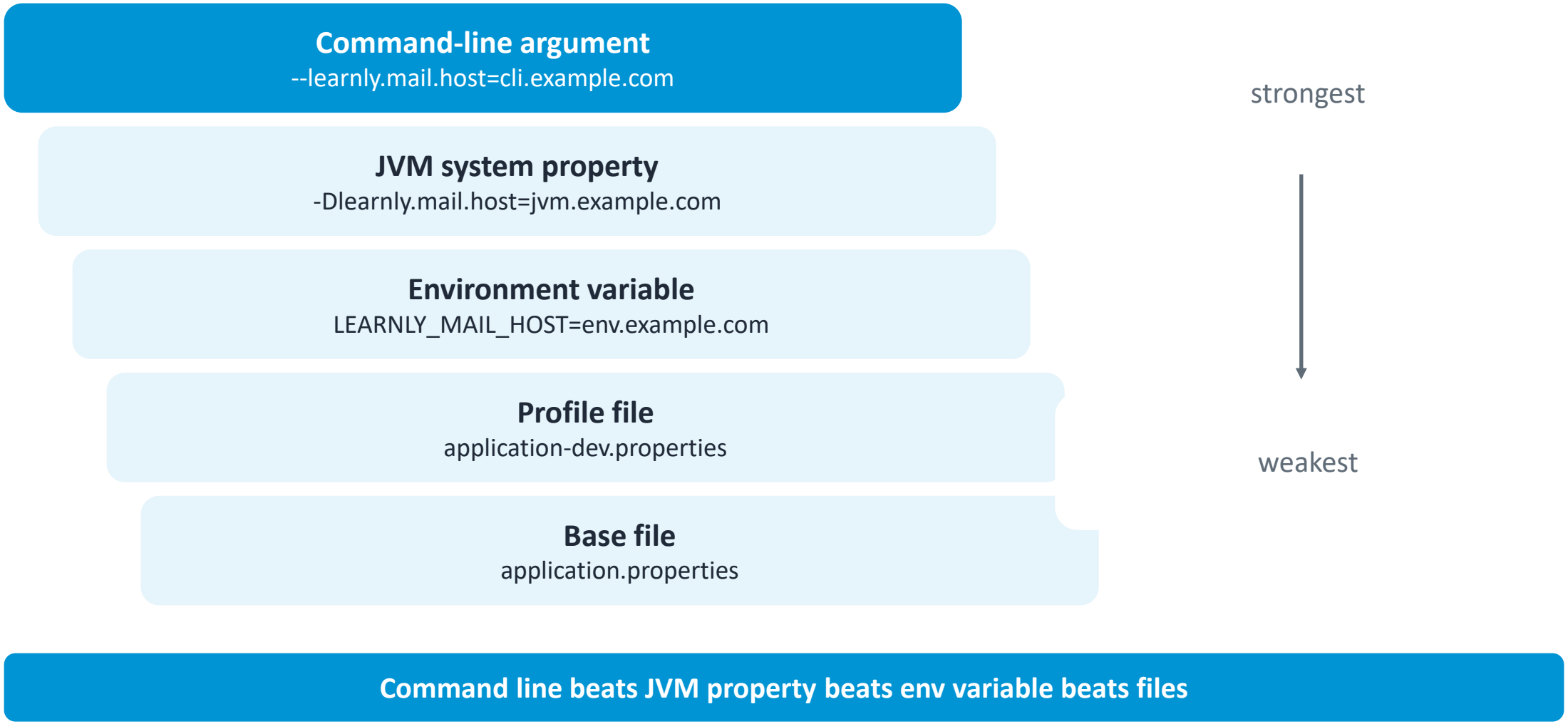
```
# application-dev.properties
learnly.datasource.url=${LEARNLY_DB_URL:jdbc:mysql://localhost:3306/learnlydb}
learnly.datasource.password=${LEARNLY_DB_PASSWORD:learnlypass}

# application-prod.properties
learnly.mail.host=${LEARNLY_MAIL_HOST:smtp.learnly.example}
learnly.datasource.password=${LEARNLY_DB_PASSWORD:}

# Refer to another property
info.app.name=${spring.application.name}
```

Developers get a working default, production supplies the variable

Who Wins? The Everyday Ladder



Environment Variables

- **Why:** every OS, container and cloud has them.
- **Names:** UPPER_CASE with underscores.
- **Mapping:** dot becomes _, dash is dropped.
- **Beats files,** loses to command line.
- **Secrets:** a common home for passwords.

```
# learnly.mail.host  ->  LEARNLY_MAIL_HOST

# bash / macOS / Linux
export LEARNLY_MAIL_HOST=env.example.com
LEARNLY_MAIL_HOST=env.example.com ./gradlew bootRun

# Windows PowerShell
$env:LEARNLY_MAIL_HOST = "env.example.com"
./gradlew bootRun

# Windows cmd
set LEARNLY_MAIL_HOST=env.example.com
```

Command-Line Args and JVM Properties

Command-line argument

Starts with two dashes, goes after the program name. Strongest everyday source. Read by Spring.

JVM system property

Starts with -D, goes before -jar. Read by Java itself, so it also affects other libraries.

```
java -jar learnly.jar --learnly.mail.host=cli.example.com      # argument
java -Dlearnly.mail.host=jvm.example.com -jar learnly.jar        # JVM property
./gradlew bootRun --args="--learnly.mail.host=cli.example.com"      # via Gradle
```

bootRun -D sets Gradle's own JVM, not your app. Use --args instead

Overriding in IntelliJ and Gradle

IntelliJ field	What to type
Active profiles	dev,inspect
Program arguments	--learnly.mail.host=cli.example.com
Environment variables	LEARNLY_MAIL_HOST=env.example.com
VM options	-Dlearnly.mail.host=jvm.example.com

Run > Edit Configurations > Modify options adds the Environment variables and VM options fields

Program arguments and VM options are different boxes - check which one

Demo: Placeholders, Order and Overrides

What we build

- **${NAME:default}:** an env variable with a fallback.
- **Four sources:** file, env, -D, argument.
- **Wrong box:** -D in Program arguments.

How the concept helps

- **Ladder:** closer to launch means stronger.
- **Proof:** /actuator/env names the winner.
- **Operators:** change values without code.

Handling Secrets

- **Secret:** password, token, API key, private key.
- **Never** in source code or Git.
- **File names the variable,** environment holds the value.
- **Better:** vault or cloud secret manager.
- **Containers:** mounted secret files (configtree).

```
# BAD  - in Git history forever
learnly.mail.password=changeme-123

# GOOD - only the name is in the file
learnly.mail.password=${LEARNLY_MAIL_PASSWORD:}

# Secret files mounted by Docker/Kubernetes
spring.config.import=optional:configtree:/run/secrets/
```

Secrets: Do and Do Not

Do

Read secrets from environment or a secret store. Use empty or dev-only defaults. Mask them in logs and Actuator. Keep .env files in .gitignore. Rotate leaked secrets at once.

Do not

Commit passwords, even in a private repository. Print records that contain secrets (override toString). Return secrets from REST endpoints. Reuse the production password in dev or test.

Dev and test get their own throw-away credentials

Demo: Handling Secrets

What we build

- **The file:** names the variable only.
- **toString:** hides the password.
- **Actuator:** shows ******.

How the concept helps

- **Git:** stays free of secrets.
- **Logs:** a stray print cannot leak.
- **Defence in depth:** three layers.

Key Takeaways

1. **Base plus profiles**
   Shared defaults in one file, differences per profile.

2. **Know the order**
   Command line beats env var beats files.

3. **Use records**
   @ConfigurationProperties with @Validated.

4. **Fail fast**
   Invalid config stops startup.

5. **Guard secrets**
   Env variables, masked in Actuator.

6. **Inspect safely**
   Expose env and configprops only in dev.



Happy Learning :)